



HAND WRITTEN - NOTES

CHAPTERS

Chapter - 1 C++ Basics

- 1.1 What is C++ ...???
- 1.2 First C++ Program
- 1.3 Variables
- 1.4 Data Types
- 1.5 Operators and Expression

Chapter - 2 Control Statements

- 2.1 Introduction
- 2.2 Conditional Statements
- 2.3 Loops
- 2.4 Switch Case Statements
- 2.5 Break and Continue

Chapter - 3 Function

- 3.1 Introduction
- 3.2 Defining - Calling a Function
- 3.3 Parameters and Arguments

- 3.4 Return Value
- 3.5 Function Overloading
- 3.6 Scope and Lifetime
- 3.7 Passing by Value
- 3.8 Passing by References
- 3.9 Default Arguments
- 3.10 Recursion

Chapter - 4 Arrays and Strings

- 4.1 Introduction
 - 4.2 Array Declaration - Initialization
 - 4.3 Accessing Array Elements
 - 4.4 Array indexing and Bounds
 - 4.5 Multidimensional Arrays
 - 4.6 String Declaration - Initialization
 - 4.7 String Manipulation
 - 4.8 Input and Output Strings
 - 4.9 String Comparison and Searching
 - 4.10 String Operation with C++ Library.
- 

Chapter -5 Pointers and References

- 5.1 Introduction
- 5.2 Declaring - Initializing Pointer
- 5.3 Accessing Pointed Data
- 5.4 Pointer Arithmetic
- 5.5 Dynamic Memory Allocation
- 5.6 Declaring and Using References
- 5.7 References VS Pointers
- 5.8 Passing by References
- 5.9 Returning References
- 5.10 References Pointers

Chapter - 6 Object-Oriented Programming

- 6.1 Introduction
- 6.2 Classes and Objects
- 6.3 Encapsulation
- 6.4 Inheritance
- 6.5 Polymorphism
- 6.6 Defining Classes .
- 6.7 Constructor and Destructor
- 6.8 Access Modifiers

- 6.9 Method Overriding
- 6.10 Abstract Class and Interfaces
- 6.11 Operator Overloading
- 6.12 Virtual Function
- 6.13 Object Composition
- 6.14 Aggregation

Chapter -7 Exception Handling

- 7.1 Introduction
- 7.2 Exception Types
- 7.3 "try-catch" blocks
- 7.4 Throwing Exceptions
- 7.5 Multiple Catch Blocks
- 7.6 RAII
- 7.7 Standard Exception Classes
- 7.8 Handling Unhandled Exceptions
- 7.9 Exception Specification

Chapter -8 File Handling

- 8.1 Introduction
- 8.2 Opening and Closing Files
- 8.3 Reading From Files

- 8.4 Writing a Files
- 8.5 Checking for file Open Success
- 8.6 Error Handling with Exceptions
- 8.7 File Pointers
- 8.8 Seeking in Files
- 8.9 Binary Files Input/output

Chapter - 9 Templates, STL

- 9.1 Introduction
- 9.2 Function Templates
- 9.3 Class Templates
- 9.4 Template Specialization
- 9.5 STL Containers
- 9.6 STL Algorithms
- 9.7 STL Iterators
- 9.8 Function Objects
- 9.9 STL Smart Pointers
- 9.10 STL Utility Function
- 9.11 STL Headers
- 9.12 STL Custom Comparators and Allocators

Chapter - 10 Multithreading

- 10.1 Introduction

- 10.2 Creating Threads
- 10.3 Thread Execution and Synchronization
- 10.4 Thread Joining and Detaching
- 10.5 Race Conditions and Data Races
- 10.6 Mutex and Locks
- 10.7 RAII and Lock Guards
- 10.8 Condition Variables
- 10.9 Thread-safe Overlaid
- 10.10 Thread Pooling

CHAPTER 1 : C++ Basics

C++ is a powerful and versatile programming language that has stood the test of time since its creation by Bjarne Stroustrup in the late 1970s. It is an extension of the C programming language with object-oriented programming features, making it suitable for both procedural and object-oriented programming paradigms.

1.1 What is C++ ?

C++ is a general-purpose programming language that combines the features of low-level languages like C with high-level features, including object-oriented programming. The "++" in C++ indicates an increment, symbolizing the language's evolution from C. C++ provides a rich set of features such as classes, inheritance, polymorphism, templates, and exception handling, making it suitable for developing complex software systems.

One of the key strengths of C++ is its efficiency

and performance. It allows for both low-level manipulation of memory and high-level abstractions, giving programmers flexibility in choosing the level of control based on the specific requirements of their applications.

1.2 First C++ Program

Lets begin our exploration of C++ by writing a simple "Hello World" program, which is a traditional starting point for learning any programming language.

Codes :-

```
#include <iostream>

int main () {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

:- Code Explanation :-

This program includes the `<iostream>` header, which provides functionality for input and output —

operations. The "main" function is the entry point of every C++ program. Here, we use "std::cout" to output the text "Hello, World" to the console. The "return 0;" statement indicates successful program execution.

1.3 Variables

In C++ variables are used to store and manipulate data. Before using a variable, you must declare its type and name. For Example :-

Codes :-

↪ int age; ↪

Declaration of an integer variable named "age".

↪ age = 25; ↪

Assignment of the value 25 to the "age" variable.

:- Variables can be of various types, including integers (int), floating-point numbers (float), characters (char), and more. They serve as — containers for data, allowing us to work with different types of information in our programs.

1.4 Data Types

In C++, data types are essential elements that define the nature of data stored in variables. They help the compiler understand how to interpret and manipulate the data during program execution. C++ supports a wide range of data types, which can be broadly categorized into primitive and derived types.

:- Primitive Data Types :-

1. int (integer)

- Represent whole numbers (positive or negative) without any decimal points.

{ int age = 25 ; }

2. Float (Floating-point)

- Represent single-precision floating-point numbers, allowing decimal points.

{ Float price = 12.99 ; }

3. double (Double-precision Floating-point)

- Similar to "float" but provides double precision, allowing decimal points for a larger range and more accurate representation of decimal values.

{ double pi = 3.14159; }

4. char (Character)

- Represent a single character, such as a letter or symbol, enclosed in single quotes.

{ char grade = 'A'; }

5. bool (Boolean)

- Represent boolean values, either "true" or "false".

{ bool isStudent = true; }

:- Derived Data Types

1. Array

- Collections of elements of the same data type,

accessed by index.

Example :-

```
int numbers[5] = {1, 2, 3, 4, 5};
```

2. Structures

- User-defined data types that group different types of data under a single name.

Example :-

```
struct Person {  
    int age;  
    char gender;  
};
```

3. Classes

- Similar to structures but with the addition of member functions, enabling object-oriented — programming concepts like encapsulation and inheritance.

Example :-

```
class Circle {
```

private :

double radius ;

public :

double getArea();

};

:- User-defined Data Types :-

In addition to the built in data types, C++ allows programmers to define their own their own data types using "typedef" and "enum". This provides flexibility and readability in code.

1. "typedef"

- Allows the creation of custom names for existing data types.

Example :-

```
typedef int Distance ;
```

```
Distance miles = 50;
```

2. enum [Enumeration]

- Defines a set of named integer constants, improving

Code readability.

Example:-

~~type~~

```
enum Day { Sunday, Monday, ..., Saturday };
```

```
Day today = Wednesday;
```

1.5 Operators

In C++, operators are symbols that perform operations on variables and values, and expressions are combinations of variables, values, and operators. Understanding how to use operators and construct expression is fundamental to performing various tasks in programming.

:- Arithmetic Operators :-

1. + (Addition) - Adds two values.

Example:-

```
int sum = 5 + 3 ; # sum is 8.
```

2. - (Subtraction) - Subtracts the right operand from the left.

Example :-

int difference = 10.4; # difference is 6.

3. * (Multiplication) - Multiplies two values.

Example :-

int product = 3 * 7; # product is 21

4. / (Division) - Divides the left operand by the right operand.

Example :-

Float quotient = 15.0 / 4.0; # quotient is 3.75

5. % (Modulo) - Return the remainder of the division.

Example :-

int remainder = 17 % 5; ↴

remainder is 2.

∘ Relational Operators ∘-

1. == (Equal to) - Checks if two values are equal.

Example ∘-

bool isEqual = (5 == 5); # isEqual is true

2. != (Not Equal to) - Checks if two values are not equal.

Example ∘-

bool notEqual = (10 != 5);
notEqual is true

3. < (Less than) - Checks if the left operand is less than the right operand.

Example ∘-

bool isLess = (3 < 8); # isLess is true

4. > (Greater than) - Checks if the left operand is greater than the right operand.

Example ∘-

bool isGreater = (12 > 7); # isGreater is true

5. <= (Less than or Equal to) - Checks if the left operand is less than or equal to the right operand.

Example:-

bool lessOrEqual = (6 <= 6); # lessOrEqual is true

6. >= (Greater than or Equal to) - Checks if the left operand is greater than or equal to the right operand.

Example:-

bool greaterOrEqual = (9 >= 9); # greaterOrEqual is true

:- Logical Operators :-

1. && (Logical AND) - Return true if both operands are true.

Example:-

bool andResult = (true && false);

andResult is false

2. || (Logical OR) - Return true if at least one operand is true.

3. ! (Logical NOT) - Returns true if the operand is false and vice versa.

Example:-

bool notResult = !(true); # notResult is false



CHAPTER = 2 : Control Statements

Control statements are fundamental elements in programming that allow developers to control the flow of execution in a program. They enable decision-making and iteration, providing the necessary flexibility to create dynamic and responsive applications.

2.1 Introduction

Control statements play a important role in programming that allow determining the order in which statements are executed. They enable the creation of complex and interactive program by allowing developers to respond to different conditions and execute specific blocks of code based on those conditions. The main types of control statements include conditional statements, loops, and switch-case statements.

2.2 Conditional Statements

Conditional statements are used to make decisions in a program. They allow the execution of different code blocks based on whether a specified condition evaluates to true or false. In C++, the most commonly used conditional statements are the "if", "else-if", and "else" statements.

• if Statements

The "if" Statements is used to execute a block of code only if a specified condition is true.

Example :-

```
int num = 10;  
  
if (num > 0) {  
    std::cout << "Positive Number" << std::endl;  
}
```

Next →

• if - else Statements

The "if-else" statement extends the "if" statement by providing an alternative block of code to execute if the condition is false.

Example :- `int num = -5;`

```
if (num > 0) {  
    std::cout << "Positive Number" << std::endl;  
} else {  
    std::cout << "Negative Number" << std::endl;  
}
```

• else-if Statements

The "else-if" statement allows the evaluation of multiple conditions in sequence.

Example :- `int num = 0;`

```
if (num > 0) {
```

```
std::cout << "Positive Number" << std::endl;  
} else if (num < 0) {  
    std::cout << "Negative Number" << std::endl;  
} else {  
    std::cout << "Number is zero" << std::endl;  
}
```

2.3 Loops

Loops are control structures that allow a block of code to be executed repeatedly. They are essential for programming or performing tasks such as iterating over arrays, processing data, and implementing algorithms. C++ supports several types of loops, including "for", "while", and "do-while" loops.

• for Loop :

The "for" Loop is commonly used for iterating over a

range of values.

Example :-

```
for (int i=1 ; i <= 5 ; ++i) {  
    std::cout << i << " " ;  
}
```

Output : 1 2 3 4 5

• While Loop

The while loop continues to execute a block of code as long as a specified condition remains true.

Example :-

```
int count = 0;  
  
while (count < 5) {  
    std::cout << count << " " ;  
    ++count ;  
}
```

Output : 0 1 2 3 4

• do-while Loop

The "do-while" loop is similar to the "while" loop, but it guarantees that the block of code is executed at least once.

Example :-

```
int count = 0;  
do {  
    std::cout << count << " ";  
    ++count;  
} while (count < 5);
```

Output : 0 1 2 3 4

2.4 Switch Case Statements

The "switch" statements provides an efficient way to handle multiple cases based on the value of an expression. It simplifies code that involves multiple "if-else if" statements.

Example :-

```
int day = 3
```

```
switch (day) {
```

```
    case 1:
```

```
        std::cout << "Monday";
```

```
        break;
```

```
    case 2:
```

```
        std::cout << "Tuesday";
```

```
        break;
```

```
    case 3:
```

```
        std::cout << "Wednesday";
```

```
        break;
```

```
    # Other Cases _____
```

```
    default:
```

```
        std::cout << "Invalid day";
```

```
}
```

- Each "case" statements represent a possible value of the expression, and the "break" statement is important to exit the "switch" block after a case is executed. If the "break" statement is omitted, the control falls through to the next case.

2.5 Break and Continue

The "break" and "continue" statements are used within loops to control their flow.

• break Statement

The "break" statement is used to exit a loop — prematurely, regardless of the loop's condition.

Example :-

```
for(int i=1; i<=10; ++i) {  
    if(i==5) {  
        break;  
    }  
    std::cout << i << " ";  
}
```

Output : 1 2 3 4

- In this example, the loop stops when "i" becomes 5 due to the "break" statement.

• continue Statement

The "continue" statement is used to skip the rest of the loop's body and proceed to the next iteration.

Example :-

```
for (int i = 1 ; i < 5 ; ++i) {  
    if (i == 3) {  
        continue ;  
    }  
    std::cout << i << " ";  
}
```

Output : 1 2 4 5

- Here, the loop skips printing the number 3 due to the "continue" statements.



CHAPTER = 3: Functions

Functions are essential building blocks in programming that allow developers to modularize code, promote code reusability, and enhance the overall structure of Program.

3.1 Introduction

A function is a self-contained block of code that performs a specific task. It is designed to be reusable and can be called from different parts of a program. Functions play an important role in structuring code, promoting maintainability, and making program more readable.

3.2 Defining - Calling a Function

In C++, a function is defined using the following syntax:-

Example :-

```
return_type function_name(parameters) {
```

Function Body

Code to perform a specific task

```
return value ; # optional return statement  
}
```

- Here :-

- "return_type" specifies the type of value the function returns.
 - "function_name" is the name given to the function.
 - "parameters" are the input values that the function can accept.
 - The function body contains the code to be executed.
 - The "return" statement, if present, returns a value to the calling code.
- ÷ To call a function, you simply use its name followed by parentheses :

Example :-

```
int result = add (5,3);
```

calling a function named "add" with arguments 5 and 3.

3.3 Parameters and Arguments

Parameters are variables listed in the function's definition, and arguments are the values passed to the function when it is called. Parameters allow functions to receive input, making them versatile and adaptable to different scenarios.

Example :-

```
int add(int a, int b) {  
    return a+b;  
}
```

∴ In this example, "a" and "b" are parameters. When calling the function, actual values (arguments) are passed in place of these parameters.

3.4 Return Value

Functions can return values using the "return" statement. The return type is specified in the function declaration.

Example :-

```
int add (int a ,int b) {  
    return a+b;  
}
```

∴ In this case, the function returns an integer value representing the sum of "a" and "b".

3.5 Function Overloading

Function overloading allows multiple functions with the same name but different parameter lists or types. This provides flexibility and enhances code readability.

Example :-

```
int add (int a ,int b) {  
    return a+b;  
}
```



```
double add(double a, double b) {
    return a+b;
}
```

÷ Here, we have two "add" functions — one for integers and one for doubles.

3.6 Scope and Lifetime

Scope refers to the region of code where a variable can be accessed. Lifetime refers to the duration for which a variable exists in memory. Variables declared within a function have local scope, and their lifetime is limited to the function's execution.

Example :-

```
void exampleFunction() {
    int x = 5; # local variable with function scope
} # _____
```

÷ Global variables, declared outside of any function, have a broader scope and a longer lifetime.

3.7 Passing by Value

When passing by value, the actual value of the variable is passed to the function. Changes to the parameter inside the function do not affect the original variable.

Example :-

```
void modifyValue (int x) {  
    x = x * 2;  
}  
  
int main() {  
    int num = 10;  
    modifyValue(num); # num remains 10  
    # .....  
}
```

3.8 Passing by Reference

Passing by reference allows a function to modify the original variable. It is done by using references or pointers.

Example:-

```
void modifyValue (int & x) {  
    x = x * 2;  
}
```

```
int main () {  
    int num = 10;  
    modifyValue (num); # num is now 20.  
    # -----  
}
```

3.9 Default Arguments

Default Arguments are specified in the function declaration. If a value is not provided for a parameter during the function call, the default value is used.

Example :-

```
int multiply (int a, int b = 2) {  
    return a * b;  
}
```

```
int result = multiply (5);
```

result is 10

3.10 Recursion

Recursion is the process where a function calls itself directly or indirectly. It allows the solution of complex problems by breaking them down into simpler instances.

Example :-

```
int factorial (int n) {  
    if (n == 0 || n == 1) {  
        return 1;  
    } else {  
        return n * factorial (n - 1);  
    }  
}
```

÷ In this example, the "factorial" function calls itself to calculate the factorial of a number.



CHAPTER - 4 : Arrays And Strings

Arrays and Strings are fundamental data structures in programming that allow the organization and manipulation of data in a structured way.

4.1 Introduction

Arrays and Strings are essential components in programming, providing a way to store and manage collection of data. An array is collection of elements of the same type, while a string is a collection of characters. Understanding their properties and operations is important for effective programming.

4.2 Array Declaration - Initialization

An array is declared by specifying its type and size. Initialization can be done at the time of declaration or later using individual assignments.

Example :-

```
int numbers[5]; ↗
```

Declaration of an integer array with size 5.

↪ # Initialization at the time of declaration

```
int scores[] = {85, 90, 78, 92, 88};
```

4.3 Accessing Array Elements

Array elements are accessed using their indices. The index starts at 0 for the first element.

Example :-

```
int scores = {85, 90, 78, 92, 88};
```

Accessing the first element (85)

```
int firstScore = scores[0];
```

Accessing the third element (78)

```
int thirdScore = scores[2];
```


4.4 Array Indexing and Bounds

Array indexing refers to the process of accessing elements using their indices. It is essential to ensure that the index is within the bounds of the array to prevent errors.

Example :-

```
int scores[] = {85, 90, 78, 92, 88};
```

Ensure index is within bounds before accessing

```
int index = 3;
```

```
if (index >= 0 && index < 5) {
```

```
    int score = scores[index];
```

```
} else {
```

Handle index out of bounds error

```
}
```

4.5 Multi dimensional Arrays

Multi dimensional arrays are arrays of arrays, allowing the creation of tables or matrices.

Example :-

```
int matrix[3][3] = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
};
```

:- Accessing element in a multidimensional array involves specifying the indices for both dimensions.

Example :-

Accessing the element in the second row, third column (6)

```
int element = matrix[1][2];
```

4.6 String Declaration - Initialization

Strings in C++ are sequences of characters. They can be declared and initialized using arrays of character.

Example :-

Declaration and initialization of a String

```
char greeting[] = "Hello, world";
```

÷ C++ also provides a "string" class in the Standard Library that simplifies string handling.

Example :-

```
#include <string>
```

```
std::string greeting = "Hello World";
```

4.7 String Manipulation

String manipulation involves various operations such as concatenation, length determination, and extraction.

Example :-

```
#include <string>
```

```
std::string greeting = "Hello"
```

```
std::string name = "Pawan";
```

Concatenation

```
std::string message = greeting + ", " + name + "!";
```


Length determination

```
int length = message.length();
```

Substring extraction

```
std::string subtring = message.substr(0,5);
```

4.8 Input and Output Strings

C++ provides ways to input and Output strings using standard input/output streams.

Example :-

```
#include <iostream>
```

```
#include <string>
```

```
int main () {
```

```
    std::string input;
```

Input string from the user

```
    std::cout << "Enter a string:";
```

```
    std::cin >> input;
```

Output the entered string

```
std::cout << "You entered: " << input << std::endl;  
return 0;  
}
```

4.9 String Comparison and Searching

String comparison involves determining the order of strings, and searching involves finding substrings within a string.

Example :-

```
#include <iostream>  
#include <string>  
  
int main(){  
    std::string str1 = "apple";  
    std::string str2 = "banana";  
  
    # String comparison  
    if (str1 == str2) {
```

```
std::cout << "Strings are equal." << std::endl;
```

```
} else {
```

```
std::cout << "Strings are not equal." << std::endl;
```

```
}
```

String Searching

```
size_t found = str1.find("le");
```

```
if (found != std::string::npos) {
```

```
std::cout << "Substring found at position" << found << std::endl;
```

```
} else {
```

```
std::cout << "Substring not found." << std::endl;
```

```
}
```

```
return 0;
```

```
}
```

4.10 String operation with C++ Standard Library

The C++ Standard Library provides a rich set of functions and methods for string manipulation,

making it easier to work with strings.

Example:-

```
#include <iostream>
```

```
#include <string>
```

```
int main () {
```

```
    std::string str = "Hello, World!";
```

```
    # Length of the String
```

```
    int length = str.length();
```

```
    # Accessing characters
```

```
    char FirstChar = str[0];
```

```
    char lastChar = str.back();
```

```
    # Substring extraction
```

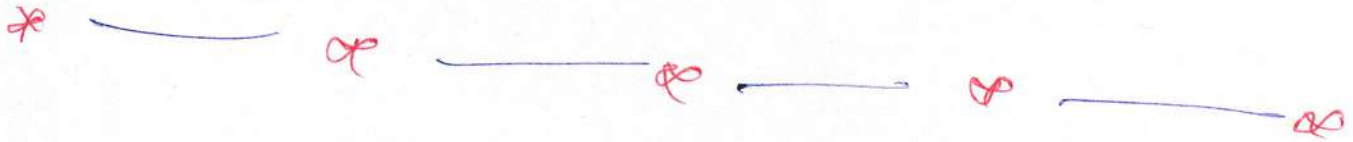
```
    std::string substring = str.substr(0, 5);
```

```
    std::cout << "Length:" << length << std::endl;
```

```
    std::cout << "First character:" << firstChar << std::endl;
```

```
std::cout << "Last Character:" << lastChar << std::endl;  
std::cout << "Substring:" << substring << std::endl;
```

```
    return 0;  
}
```



CHAPTER : 5 : ^{and} ↑ Pointers - References

Pointers and references are powerful features in C++ that provide ways to manipulate memory and work with data more efficiently.

5.1 Introduction

Pointers and references are advanced features in C++ that allow for more directly control over memory and provide efficient ways to work with data. They are fundamentals in dealing with dynamic memory allocation and enhancing the flexibility of functions and data manipulation.

5.2 Declaring - Initializing Pointer

A pointer is a variable that stores the memory address of another variable. Declaring and ---

initializing a pointer involves specifying the type of the variable it points to and assigning the address of that variable.

Example :-

```
int number = 10;
```

Declaration of a pointer to an integer

```
int *ptr;
```

Initialization with the address of "number"

```
ptr = &number;
```

5.3 Accessing Pointed Data

To access the data pointed to by a pointer, we use the dereference operator (*).

Example :-

```
int number = 10
```

Initialization with the address of "number"

```
int *ptr = &number;
```

Accessing the value pointed to by "ptr"

int value = *ptr

5.4 Pointer Arithmetic

Pointer arithmetic allows manipulation of pointers using addition and subtraction operations. This is often used when dealing with arrays.

Example :-

int number[] = {1, 2, 3, 4, 5};

Initialization with the address of the first element

int *ptr = number;

Accessing the third elements using pointer Arithmetic

int thirdElement = *(ptr + 2);

5.5 Dynamic Memory Allocation

Dynamic memory allocation allows us to allocate memory at runtime using pointers. The "new" operator is used for dynamic memory allocation, and the "delete" operator is used to free the allocated memory.

Example:-

Dynamic allocation of memory for an integer

```
int *ptr = new int;
```

Assigning a value to the allocated memory

```
*ptr = 42;
```

Freeing the allocated memory

```
delete ptr;
```

÷ Dynamic memory allocation is particularly useful when dealing with unknown or changing amounts of data.

5.6 Declaring and Using References

References provide an alias for an existing variable. They are declared using the "&" symbol.

Example :-

```
int number = 10;
```

± Declaration and initialization of a reference

- References must be initialized when declared and cannot be changed to refer to another variable.

5.7 References VS Pointers

While pointers and references share similarities, they also have key differences :-

- Pointers can be reassigned to point to different variables, whereas references cannot be changed once initialized.

- Pointers can be set to "nullptr" (or "NULL"), indicating they do not point to any valid memory, while references always refer to a valid object.
- References automatically dereference, while pointers need explicit dereferencing using "*" .

5.8 Passing by References

Passing by reference in function parameters allow the function to directly manipulate the original data.

Example :-

```
void modifyValue (int &x) {  
    x = x * 2 ;  
}
```

```
int main () {  
    int num = 10 ;
```

```
modifyValue(num); # num is now 20
```

```
} # - - - - -
```

∴ This avoids the need to use pointers and provides a cleaner syntax.

5.9 Returning References

Functions can return references, enabling them to return values that can be modified directly.

Example:-

```
int numbers[] = {1, 2, 3, 4, 5};  
int& getElement (int index) {  
    return numbers[index];  
}
```

```
int main () {
```

```
    getElement(2) = 10; # Set the value at index 2 to 10.
```

}

5.10 Reference Pointers

A reference pointer is a combination of a reference and a pointer. It is declared using both "*" and "&" and allows for pointer-like behaviors with reference syntax.

Example :-

```
int number = 42;
```

Pointer to "number"

```
int *ptr = &number;
```

Reference to the value pointed to by "ptr"

```
int &ref = *ptr;
```

- Reference pointers are particularly useful when dealing with functions that return pointers.

* _____ ✂ _____ ✂ _____ ✂ _____ ✂

CHAPTER = 6 : Object-Oriented Programming

Object - Oriented Programming (OOP) is a paradigm that uses objects and classes to structure code, programming promoting modularity, reusability, and flexibility.

6.1 Introduction

Object - Oriented Programming (OOP) is a programming paradigm that organizes code into objects, which are instances of classes. It revolves around the principles of encapsulation, inheritance and polymorphism, providing a modular and efficient way to design and structure code.

6.2 Classes and Objects

- Classes :- Classes are user-defined data types that encapsulate data and behavior. They serve as blueprints for creating objects. A class defines the properties

(attributes) and methods (functions) that its object will have.

Example :-

```
class Car {  
    public:  
        # Properties  
        std::string brand;  
        int year;  
        ## Method  
        void startEngine () {  
            # Code to start the engine  
        }  
};
```

- Objects :- Objects are instances of classes. They represent real-world entities and have specific attributes and behaviors defined by their class.

Example :-

```
# Creating an object of the Car class  
Car myCar;
```



```
myCar.brand = "Toyota";  
myCar.year = 2022;  
myCar.startEngine();  
...
```

6.3 Encapsulation

Encapsulation is the bundling of data and methods that operate on the data into a single unit, i.e., a class. It hides the internal details of how the object works and exposes only what is necessary.

Example :-

```
class Circle {  
    private :  
        double radius;  
    public :  
        # Setter method  
    void setRadius (double r) {  
        if (r > 0) {  
            radius = r;  
        }  
    }  
}
```

```
}  
}
```

Getter Method

```
double getArea() {  
    return 3.14 * radius * radius;  
}
```

÷ In this example, "radius" is encapsulated and can only be accessed or modified through the public methods of the class.

6.4 Inheritance

Inheritance is a mechanism where a new class (derived or child class) inherits properties and behaviors from an existing class (base or parent class). It promotes code reuse and establishes an "is-a" relationship between classes.

Example :-

```
class Animal {  
    public :  
    void eat () {
```

Code for eating

```
    }  
};
```

```
class Dog : public Animal {  
    public :  
    void bark () {
```

Code for barking

```
    }  
};
```

∴ The "Dog" class inherits the "eat" method from the "Animal" class.

6.5 Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common base class.

It includes both compile-time (function overloading) and runtime (function overriding) polymorphism.

Example :-

```
class Shape {  
    public:  
    virtual void draw () {  
        # Base implementation  
    }  
};
```

```
class Circle : public Shape {  
    public:  
    void draw() override {  
        # Code to draw a circle  
    }  
};
```

```
class Square : public Shape {  
    public:  
    void draw () override {  
        # Code to draw a Square  
    }  
};
```

6.6 Defining Classes

Defining a class involves specifying its attributes and behaviors. Attributes are often private, and public methods are used to interact with or modify those attributes.

Example :-

```
class Person {
```

```
    private:
```

```
        std::string name;
```

```
        int age;
```

```
    public:
```

```
        # Constructors
```

```
        Person (std::string n, int a): name(n), age(a) {}
```

```
        # Getter Methods
```

```
        std::string getName () const {
```

```
            return name;
```

```
        }
```

```
int getAge () const {
    return age;
}
```

Setter Method

```
void SetAge (int a) {
    if (a > 0) {
        age = a;
    }
}
...

```

6.7 Constructor and Destructor

- Constructor :- A Constructor is a special method that is called when an object is created. It initializes the object's properties and allocates resources.

Example :-

```
class Book {
private :
    std::string title;
```


public :

Constructors

```
Book (std::string t) : title (t) {
```

```
    std::cout << "Book created." << title << std::endl;
```

```
}
```

Other Methods

```
};
```

- Destructor :- A destructor is called when an object goes out of scope. It is responsible for releasing resources and cleaning up.

Example :-

```
class Book {
```

```
    private :
```

```
        std::string title;
```

```
public :
```

Constructors

```
Book (std::string t) : title(t) {
```

```

std::cout << "Book created:" << title << std::endl;
}

# Destructor
~ Book () {
    std::cout << "Book destroyed:" << title << std::endl;
}

# Other Methods
};

```

6.8 Access Modifiers

Access modifiers ("public", "private", and "protected") control the visibility of class members.

- Public :- Members are accessible from outside the class.
- Private :- Members are only accessible within the class.
- Protected :- Members are accessible within the class and its derived class.

Example :-

```
class Example {  
    public :  
        # Public Member  
        int publicVar;  
    private :  
        # Private Member  
        int privateVar;  
    protected :  
        # Protected member  
        int protectedVar;  
};
```

6.9 Method Overriding

Method Overriding occurs when a derived class provides a specific implementations for a method that is already defined in its base class.

Example :-

```
class Animal {
```



```

public:
virtual void makeSound () {
    std::cout << "Generic animal Sound" << std::endl;
}
};

class Dog : public Animal {
public:
    void makeSound () override {
        std::cout << "Bark! Bark!" << std::endl;
    }
};

```

6.10 Abstract Classes and Interfaces

Abstract classes contain one or more abstract (pure virtual) methods, which are meant to be overridden by derived classes. An interface is a class with only pure virtual methods.

Example :-

```
class Shape {
```

```
public :
```

```
# Pure Virtual Method
```

```
virtual void draw () const = 0 ;
```

```
};
```

```
class Circle : public Shape {
```

```
public :
```

```
void draw () const override {
```

```
# implementation for drawing a circle
```

```
}
```

```
};
```

6.11 Operator Overloading

Operator overloading allows the definition of custom behavior for operators.

Example:-

```
class Complex {
```

```
private :
```

```
double real ;
```

```
double imaginary ;
```

public:

```
Complex(double r, double i): real(r), imaginary(i) {}
```

Overloading the + operator

```
Complex operator+(const Complex& other) const {  
    return Complex(real + other.real, imaginary + other.  
        imaginary);  
}  
};  
...
```

6.12 Virtual Function

A virtual function is a member function that is declared within a base class and redefined by a derived class. It enables dynamic polymorphism.

Example :-

```
class Shape {  
    public:  
        virtual void draw () const {
```


Base implementation

```
}  
};
```

```
class Circle : public Shape {  
    public:
```

```
    void draw () const override {
```

implementation for drawing a circle

```
}  
};
```

6.13 Object Composition

Object composition involves combining different classes to create a new class with more complex functionality.

Example :-

```
class Engine {
```

```
    public:
```

```
    void start () {
```

Code to start the engine

```
}  
};
```

```

class Car {
    private:
        Engine carEngine;

    public:
        void startCar () {
            carEngine.start ();
            # Additional code to start the car
        }
};

```

6.14 Aggregation

Aggregation is a form of object composition where one class contains another class, but the contained class can exist independently.

Example:-

```

class Department {
    # Department Details
};

```

```
class Employee {  
    private :
```

```
        Department* empDepartment ;
```

```
    public :
```

```
        Employee(Department* dept) :
```

```
            empDepartment(dept) {}
```

```
    ## Other methods -
```

```
};
```

* ← ∞ — ∞ — ∞ — ∞ — ∞ — ∞ — ∞

CHAPTER - 7 : Exception Handling

Exception handling is a critical aspects of programming that allows developers to manage errors and unexpected situations in a systematic way.

7.1 Introduction

Exception handling is a programming paradigm that deals with the occurrence of unexpected events, often referred to as exceptions, during the execution of a program. These events may include runtime errors, such as dividing by zero, accessing an invalid memory location, or file I/O errors. Exception handling provides a mechanism to gracefully handle such situations and prevent program crashes.

7.2 Exception Types

Exception can be broadly categorized into types:-

- System Exception :- These are exception thrown by the runtime system or underlying libraries. Example include memory access violations, arithmetic overflows, and stack overflow.
- User-Defined Exception :- These are exceptions explicitly thrown by the programmer to include or indicate exceptional conditions in the program.

Example :-

User-defined exception

```
class MyException : public std::exception {
public:
    const char* what() const noexcept override {
        return "This is custom Exception";
    }
};
```

7.3 "try-catch" Blocks

The "try-catch" block is a fundamental construct in exception handling. The "try" block encloses the code that might throw an exception, and the "catch" block handles the

exception if it occurs.

Example:-

```
try {
```

Code that may throw an exception

```
} catch (const std::exception & e) {
```

Handle the exception

```
std::cerr << "Exception caught : " << e.what() << std::endl;
```

```
} catch (...) {
```

Catch all blocks for handling any other type of exception

```
std::cerr << "Unknown exception caught" << std::endl;
```

```
}
```

...

7.4 Throwing Exception

Exceptions are thrown using the "throw" Keyword. This interrupts the normal flow of the program and transfers control to the nearest suitable catch block.

Example:-

```
int divide (int a, int b) {
```

```
if (b == 0) {
```

Throwing an exception for division by zero


```

        throw std::runtime_error("Division by Zero");
    }
    return a/b;
}
...

```

7.5 Multiple Catch Blocks

A single "try" blocks can have multiple "catch" blocks to handle different types of exceptions.

Example :-

```

try {
    // Code that may throw an exception
} catch (const std::runtime_error& e) {
    // Handle runtime error
} catch (const std::logic_error& e) {
    // Handle logic error
} catch (const std::exception& e) {
    // Handle other exception
} catch (...) {

```

Catch all block

}
...

7.6 RAII (Resource Acquisition is Initialization)

RAII is a programming idiom that ties the lifecycle of a resource to the scope of an object. It ensures that resources (such as memory, files, or network connections) are properly acquired and released.

Example:-

```
class FileHandler {  
    private:  
        FILE* file ;
```

```
    public:
```

```
        explicit FileHandler (const char* filename):
```

```
            file (std::fopen (filename, "r")) {
```

```
        if (!file) {
```

#Throwing an exception if file opening file

```
            throw std::runtime_error ("Failed to open file");
```

```
        }
```

File Handler () {

std::fclose(file);

Automatically closes the file when the object goes out of scope

}

Other methods ...

};

7.7 Standard Exception Classes

The C++ standard Library provides a set of standard exception classes derived from "std::exception". These includes —

"std::runtime_error", "std::logic_error", —

"std::invalid_argument", and others.

Example:-

try {

Code that may throw an exception

} catch (const std::runtime_error& e) {

Handle runtime error

} catch (const std::logic_error& e) {

Handle logic error

```
} catch (const std::exception& e) {
```

Handle other exception

```
} catch (...) {
```

Catch all block

```
}
```

...

7.8 Handling Unhandled Exceptions

Unhandled exception can lead to program termination. To handle unhandled exceptions, we can use the "std::set_terminate" function.

Example :-

```
void custom Terminate () {  
    std::cerr << "Unhandled exception. Terminating program"  
               << std::endl;  
    std::exit (EXIT_FAILURE);  
}  
int main () {  
    std::set_terminate (custom Terminate);
```

```
    # Rest of the Program  
}  
...
```

7.9 Exception Specification

Exception specification (throw clauses) are part of a function's declaration and indicate the types of exceptions it can throw. However, their usage is discouraged in modern C++.

Example :-

```
void myfunction() throw(std::runtime_error) {  
    # Code that may throw a runtime-error  
}  
...
```

- In modern C++, it is recommended to use "noexcept" specifier and "std::nothrow" for all memory allocation.

Example :-

```
void myfunction() noexcept {  
    # Code that does not throw exception  
}  
...
```

CHAPTER = 8 : File Handling

File handling is an important aspect of programming that allows for reading from and writing to files.

8.1 Introduction

File handling in C++ provides a way to interact with external files, allowing programs to persistently store and retrieve data. This is essential for tasks like data storage, configuration settings, and communication between programs.

8.2 Opening and Closing Files

To perform file operations, a file must first be opened.

The "std::fstream" class is commonly used for file handling.

Example:-

```
#include <fstream>

int main () {
    std::fstream file;
```



```
# Open for writing  
file.open("example.txt", std::ios::out);
```

```
if ( file.is_open() ) {
```

```
# File operations
```

```
file.close(); # Close the file when done
```

```
} else {
```

```
# Handle file opening failure
```

```
}
```

```
return 0;
```

```
}
```

```
...
```

8.3 Reading From Files

Reading from files involves using input file streams (—— "std::ifstream"). The ">>" operator is commonly used for extracting data from a file.

Example :-

```
#include <fstream>
```

```
#include <iostream>
```

```
int main () {
```

```
std::ifstream inputFile ("example.txt");
```

```
if (inputFile.is_open()) {
```

```
    int value;
```

```
    while (inputFile >> value) {
```

```
        std::cout << "Read: " << value << std::endl;
```

```
    }
```

```
    inputFile.close();
```

```
} else {
```

```
    # Handle file opening failure
```

```
}
```

```
    return 0;
```

```
}
```

```
...
```

8.4 Writing to files

Writing to files involves using output file stream (i.e., "std::ofstream"). The "<<" operator is commonly used for inserting data into a file.

Example :-

```
#include <fstream>

int main () {
    std::ofstream outputFile("example.txt");

    if (outputFile.is_open()) {
        outputFile << "Hello, File!" << std::endl;
        outputFile.close();
    } else {
        # Handle file opening failure
    }
    return 0;
}

...
```

8.5 Checking for File Open Success

Checking whether a file has been successfully opened is important to avoid issues with a subsequent file operations. The "is-open()" member function of File streams is used for this purpose.

Example :-

```
#include <fstream>

int main() {
    std::fstream file;
    file.open("example.txt", std::ios::out);

    if (file.is_open()) {
        # File operations
    } else {
        # Handle file opening failure
    }
    return 0;
}
```

8.6 Error Handling with Expression

Error handling in file operations can be improved using exception handling. This ensures that exceptions are caught and handled gracefully.

Example :-

```
#include <fstream>
#include <iostream>
```

```
int main () {
```

```
    try {
```

```
        std::ifstream inputFile ("example.txt");
```

```
        if (!inputFile.is_open()) {
```

```
            throw std::runtime_error("Failed to open file");
```

```
        }
```

```
        # File operations
```

```
        inputFile.close();
```

```
    } catch (const std::exception& e) {
```

```
        std::cerr << "Exception:" << e.what() << std::endl;
```

```
    }
```

```
    return 0;
```

```
}
```

```
...
```

8.7 File Pointers

File pointers help keep track of the current position in a file. They can be moved using functions like "seekg" and "seekp".

Example :-

```
#include <fstream>
```

```
#include <iostream>
```

```
int main () {
```

```
std::fstream file ("example.txt", std::ios::in);
```

```
if (file.is_open()) {
```

```
    # Move 2 position from the beginning
```

```
    file.seekg (2, std::ios::beg);
```

```
    int value;
```

```
    file >> value;
```

```
    std::cout << "Read:" << value << std::endl;
```

```
    file.close();
```

```
} else {
```

```
    # Handle file opening failure
```

```
}
```

```
return 0;
```

```
}
```

...

8.8 Seeking in Files

Seeking in files is the process of moving the file pointer to a specific position. This is useful for random access for updating specific positions of a file.

Example :-

```
#include <fstream>
#include <iostream>

int main () {
    std::fstream file("example.txt", std::ios::in | std::ios::out);
    if (file.is_open()) {
        # Move the put pointer to position 5 from the beginning
        file.seekp(5, std::ios::beg);
        # Overwrite content at position 5.

        file.close();
    } else {
        # Handle file opening failure
    }
}
```

```
    return 0;  
}  
...
```

8.9 Binary File Input/Output

Binary file input/output involves reading and writing raw binary data. This is useful for handling non-text files, such as images or executable files.

Example :-

```
#include <fstream>  
  
struct Data {  
    int value;  
    double factor;  
};  
  
int main() {  
    std::ofstream binaryfile("data.bin", std::ios::binary);  
    if (binaryfile.is_open()) {  
        Data data = {42, 3.14};  
    }
```

```
binaryFile.write(reinterpret_cast<const char*>(&data),  
sizeOf(Data));
```

```
binaryFile.close();
```

```
} else {
```

```
    # Handle file opening failure
```

```
}
```

```
return 0;
```

```
}
```

...

⇒ Reading binary data involves a similar process, using "read" instead of "write".

Example:-

```
#include <fstream>
```

```
#include <iostream>
```

```
struct Data {
```

```
    int value;
```

```
    double factor;
```

```
};
```



```
int main() {
```

```
std::ifstream binaryFile("data.bin", std::ios::binary);
```

```
if (binaryFile.is_open()) {
```

```
    Data data;
```

```
    binaryFile.read(reinterpret_cast<char*>(&data), sizeof(Data));
```

```
    std::cout << "Read: " << data.value << ", " << data.factor <<
    std::endl;
```

```
    binaryFile.close();
```

```
} else {
```

~~## Handle file opening failure~~

```
}
```

```
    return 0;
```

```
}
```



CHAPTER = 9 : Templates, STL

Template
Standard Library

9.1 Introduction

Templates in C++ provide a powerful mechanism for writing generic code that can work with different data types. The standard Template Library (STL) is a collection of template classes and functions that extends the capabilities of C++.

9.2 Function Templates

Function templates allow writing functions that work with any data type. The template parameter is represented by the keyword "typename" or "class".

Example :-

```
template <typename T>
```

```

T add (Ta, Tb) {
    return a+b;
}

int main () {
    # Calls add <int>
    int sumint = add(5, 10);

    # Calls add <double>
    double sumDouble = add(3.5, 2.7);

    # -----
}
...

```

9.3 Class Templates

Class templates enable the creation of generic classes that can work with different data types.

Example:-

```

template <typename T>
class Pair {
    private:
        T first;
        T second;

```


public :

Pair (T_f, T_s) : First(f), second(s) { }

T getFirst () const {
 return first;
}

T getSecond () const {
 return second;
}
};

...

9.4 Template Specialization

Template specialization allows providing a specific implementation for a particular set of template arguments.

Example :-

```
template <typename T>  
class MyContainer {
```

```
# Generic implementation  
};
```

```
class MyContainer<char> {
```

```
# Specialized implementation for char  
};
```

```
...
```

9.5 STL Containers

STL containers are template classes that provide dynamic arrays, linked lists, queues, stacks and more.

Example :-

```
#include <vector>
```

```
#include <list>
```

```
int main () {
```

```
std::vector<int> vec; # Dynamic array
```

```
std::list<double> list; # Double linked list
```

```
# -----
```

```
}
```

```
...
```

9.6 STL Algorithms

STL algorithms operate on container and perform various operations like sorting, searching, and modifying elements.

Example :-

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> vec = {5, 2, 8, 1, 7};
    std::sort(vec.begin(), vec.end());
    # Sorts the vector
    # _____
}
...
```

9.7 STL Iterators

STL iterators provide a way to traverse elements of a container. They are similar to pointers.

Example :-

```
#include <vector>
```



```
int main () {
```

```
std::vector<int> vec = {1, 2, 3, 4, 5};
```

```
for (auto it = vec.begin(); it != vec.end(); ++it) {
```

```
    # Access elements using iterators
```

```
    # _____
```

```
}
```

```
...
```

9.8 Function Objects

Function objects (functions) are objects that can be called as if they were functions. They are often used with STL algorithms.

Example:-

```
#include <vector>
```

```
#include <algorithm>
```

```
struct Sqr {
```

```
    int operator() (int x) const {
```

```

        return x * x;
    }
};

int main () {
    std::vector<int> vec = {1, 2, 3, 4, 5};
    std::transform(vec.begin(), vec.end(), vec.begin(),
                   square);
}
...

```

9.9 STL Smart Pointers

STL smart pointers ("std::unique_ptr", "std::shared_ptr", "std::weak_ptr") manage the memory of dynamically allocated objects.

Example :-

```

#include <memory>

int main () {
    std::unique_ptr<int> unique_ptr = std::make_unique
    <int>(42);
}

```

```
std::shared_ptr<int> sharedPtr = std::make_shared  
    <int>(42);
```

```
    #  
}  
...
```

9.10 STL Utility Functions

STL provides utility functions like "std::pair", "std::make_pair", "std::swap", and "std::move".

Example:-

```
#include <utility>  
  
int main() {  
    std::pair<int, double> myPair(5, 3.14);  
    std::swap(myPair.first, myPair.second);  
    #  
}  
...
```

9.11 STL Headers

STL headers include various template classes

and functions. Common headers include "<vector>", "list", "<algorithm>", "<iterator>", and "<memory>".

Example :-

```
#include <vector>
#include <algorithm>
#include <iterator>

int main() {
    # Using STL Headers
    std::vector<int> vec = {1, 2, 3};
    std::sort(vec.begin(), vec.end());
    std::copy(vec.begin(), vec.end(),
              std::ostream_iterator<int>(std::cout, " "));
    # -----
}
```

9.12 STL Custom Comparators and Allocators.

STL containers and algorithms often allow customization through custom comparison or comparators and allocators.

Example :-

```
#include <vector>
```

```
#include <algorithm>
```

```
bool customComparator(int a, int b) {
```

```
    # Custom comparison logic
```

```
    return a > b;
```

```
}
```

```
int main() {
```

```
    std::vector<int> vec = {5, 2, 8, 1, 7};
```

```
    std::sort(vec.begin(), vec.end(), customComparator);
```

```
    # _____
```

```
}
```

...

~~✗~~ — ~~✗~~ — ~~✗~~ — ~~✗~~ — ~~✗~~ — ~~✗~~

CHAPTER = 10 : Multithreading

10.1 Introduction

Multithreading is a programming paradigm that involves the concurrent execution of multiple threads in a single process. Threads allow for parallelism, enabling programs to perform multiple tasks simultaneously.

10.2 Creating Threads

Creating threads in C++ involves using the `<thread>` header. Threads can execute functions as callable objects concurrently.

Example:-

```
#include <iostream>
#include <thread>
```



```
void myFunction () {
    # Code to be executed by the thread
    std::cout << "Thread executing." << std::endl;
}
```

```
int main () {
    # Create a thread and pass the function
    std::thread myThread (myFunction);

    # Wait for the thread to finish
    myThread.join();

    return 0;
}
...
```

10.3 Thread Execution and Synchronization

Thread execution is concurrent, and synchronization mechanism are necessary to co-ordinate access to shared resources. Common synchronization tools include mutexes and locks.

Example:-

```
#include <iostream>
#include <thread>
#include <mutex>
```

```
std::mutex myMutex;
```

```
void sharedResourceFunction () {
```

RAII lock guard.

```
std::lock_guard<std::mutex> lock(myMutex);
```

Code accessing shared Resource

```
std::cout << "Thread accessing shared Resource" << std::endl;
```

```
}
```

```
int main () {
```

```
std::thread thread1(sharedResourceFunction);
```

```
std::thread thread2(sharedResourceFunction);
```

```
thread1.join();
```

```
thread2.join();
```

```
return 0;
```

```
}
```

10.4 Thread Joining and Detaching

Thread joining ensures that the main thread waits for a child thread to finish before proceeding.

Detaching allows a thread to run independently.

Example:-

```
#include <iostream>
```

```
#include <thread>
```

```
void independentFunction() {
```

Code executed independently

```
std::cout << "Independent thread" << std::endl;
```

```
}
```

```
void dependentFunction() {
```

Code dependent on other threads

```
std::cout << "Dependent thread" << std::endl;
```

```
}
```

```
int main() {
```

```
std::thread independentThread(independentFunction);
```

```
std::thread dependentThread(dependentFunction);
```


Wait for independent Thread to finish
independentThread.join();

Let dependent Thread run independently
dependentThread.detach();

return 0;

}

...

10.5 Race conditions and Data Races

Race condition occur when multiple threads access shared data concurrently, leading to unpredictable results. Data races are a specific type of race condition involving unsynchronized access to shared data.

Example :-

```
#include <iostream>
#include <thread>
```

```
int sharedData = 0;
```

```
void raceConditionFunction() {
```

```
    for (int i = 0; i < 100000; ++i) {
```

```
sharedData++; # Potential race condition
```

```
}  
}
```

```
int main () {
```

```
    std::thread thread1(raceConditionFunction);
```

```
    std::thread thread2(raceConditionFunction);
```

```
        thread1.join();
```

```
        thread2.join();
```

```
    std::cout << "Shared Data:" << sharedData << std::endl;
```

```
    return 0;
```

```
}
```

```
...
```

10.6 Mutex and Locks

Mutexes (mutual exclusion) are synchronization primitives that prevent multiple threads from simultaneously accessing shared resources. Locks as "std::lock_guard", simplify mutex usage

.....

Example :-

```
#include <iostream>
```

```
#include <thread>
```

```
#include <mutex>
```

```
std::mutex myMutex;
```

```
void synchronizedFunction () {
```

```
    # Lock guard for automatic locking and unlocking
```

```
    std::lock_guard<std::mutex> lock(myMutex);
```

```
    # Code accessing shared resources
```

```
    std::cout << "Thread Accessing" << std::endl;
```

```
}
```

```
int main () {
```

```
    std::thread thread1(synchronizedFunction);
```

```
    std::thread thread2(synchronizedFunction);
```

```
    thread1.join();
```

```
    thread2.join();
```

```
    return 0;
```

```
}
```


10.7 RAII and Lock Guards

RAII (Resources Acquisition is initialization) is a design pattern where resources management is tied to object lifetime. Lock guards are an example of RAII for managing ... mutexes.

Example :-

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex myMutex;

void synchronizedFunction () {
    #RAII lock guard
    std::lock_guard <std::mutex> lock(myMutex);

    #Code accessing shared Resources
    std::cout << "Thread accessing " < std::endl;

    # Lock is automatically released when lock
    goes out of scope
}
```

```

int main () {
    std::thread thread1(synchronizedFunction);
    std::thread thread2(synchronizedFunction);

    thread 1.join();
    thread 2.join();

    return 0;
}
...

```

10.8 Condition Variables

Condition variables allow threads to synchronize their execution based on the state of shared data.

Example:-

```

#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variables>

std::mutex myMutex;
std::condition_variable myConditionVariable;

```

```
bool dataReady = false;
```

```
void waitforData () {
```

```
    std::unique_lock<std::mutex> lock(myMutex);
```

```
    # Wait until data is ready
```

```
    myConditionVariable.wait(lock, [] { return dataReady; });
```

```
    # Code to process data
```

```
    std::cout << "Data is ready" << std::endl;
```

```
}
```

```
void prepareData () {
```

```
{
```

```
    std::lock_guard<std::mutex> lock(myMutex);
```

```
    dataReady = true;
```

```
}
```

```
    # Notify the waiting thread that data is ready
```

```
    myConditionVariable.notify_one();
```

```
}
```

```
int main () {
```

```
    std::thread thread1(waitforData);
```



```
std::thread thread2 (prepareData);
```

```
thread1.join();
```

```
thread2.join();
```

```
return 0;
```

```
}
```

```
...
```

10.9 Thread-Safe Queues

Ensuring thread safety involves protecting shared data with mutexes or other synchronization mechanisms.

Example :-

```
#include <iostream>
```

```
#include <mutex>
```

```
##
```

```
class Counter {
```

```
private :
```

```
int value ;
```

```
std::mutex valueMutex;
```

```
public :
```

```
Counter() : value(0) {}
```

```
...
```

```
void increment () {
```

```
    std::lock_guard<std::mutex> lock(valueMutex);
```

```
    return value;
```

```
}
```

```
};
```

```
int getvalue () const {
```

```
    std::lock_guard<std::mutex> lock(valueMutex);
```

```
    return value;
```

```
}
```

```
};
```

```
int main () {
```

```
    Counter counter;
```

```
    std::thread thread1 (&counter) {
```

```
        for (int i=0; i<1000000; ++i) {
```

```
            counter.increment();
```

```
        }
```

```
    };
```

```
    std::thread thread2 (&counter) {
```

```
        for (int i=0; i<100000; ++i) {
```

```

        counter.increament ();
    }
};

```

```

thread 1.join ();

```

```

thread 2.join ();

```

```

std::cout << "Counter Value" << counter.getvalue ()
<< std::endl;

```

```

return 0;

```

```

}

```

...

10.10 Thread Pooling

Thread pooling involves creating a group of reusable threads to improve performance.

Example :-

```

#include <iostream>
#include <thread>
#include <vector>
#include <queue>
#include <Functional>
#include <mutex>
#include <condition_variable>

```



```
class ThreadPool {
```

```
private :
```

```
std::vector<std::thread> threads;
```

```
std::queue<std::function<void()>> tasks;
```

```
std::mutex taskMutex;
```

```
std::condition_variable taskCondition;
```

```
bool stop;
```

```
public :
```

```
explicit ThreadPool(size_t threadCount) : stop(false) {
```

```
for (size_t i = 0; i < threadCount; ++i) {
```

```
threads.emplace_back([this] {
```

```
while (true) {
```

```
std::function<void()> task;
```

```
{
```

```
std::unique_lock<std::mutex> lock(taskMutex);
```

```
taskCondition.wait(lock, [this] { return stop ||  
!tasks.empty(); });
```

```
if (stop && tasks.empty()) {
```

return 0;

}

task = std::move(task.front());

task.pop();

}

task();

}

});

}

}

template < typename T >

void enqueueTask(T task) {

{
std::unique_lock<std::mutex> lock(taskMutex);

task.emplace(task);

}

taskCondition.notify_one();

}

~ Thread Pool () {

{

std::unique_lock<std::mutex> lock(taskMutex);

```
    stop = true;  
}
```

```
taskCondition.notifyAll();
```

```
for (std::thread & thread : threads) {
```

```
    thread.join();
```

```
}
```

```
}
```

```
};
```

```
int main() {
```

```
    ThreadPool pool(4);
```

```
    for (int i = 0; i < 8; ++i) {
```

```
        pool.enqueueTask([i] {
```

```
            std::cout << "Task " << i << "executing" << std::endl;
```

```
        });
```

```
    }
```

```
    return 0;
```

```
}
```

...